

Second Individual Assessment

 Published

 Assign To

 Edit



COMS W4156 Advanced Software Engineering (ASE)

Second Individual Assessment Fall 2023

This is an individual assignment. You are NOT permitted to post, discuss or work together on the assessment questions with other students in the class or with anyone else other than the teaching staff. Your answers can be as long or as short as you want, but a long meandering answer that incidentally mentions something relevant will receive less credit than a short high-quality answer. Submit your answers all in a single document in response to this assignment on courseworks. You can submit as many times as you want until the deadline. Submissions via email, edstem, or any other means besides courseworks will not be graded and will receive zero credit.

Academic Honesty policy: All submissions will be automatically analyzed by turnitin.com for plagiarism/collusion. Plagiarism or collusion will result in zero credit on the assessment and referral to the relevant disciplinary committee.

Late policy: Submissions posted one day late (which includes everything from 1 minute late to 23 hours 59 minutes late) will receive maximum 75% credit; two days late maximum 50% credit; three days late maximum 25% credit; four days late 0 credit.

Maximum total 100 points. There is no credit for part 0, but 0A is required in the sense that we will not grade the graded questions if part 0A is missing or incomplete and you will receive zero credit for the assessment. Part 0B is optional and will not affect your grade.

Ungraded questions: part 0A is required and part 0B is optional.

part 0A (required): You have 100 total points to allocate among the members of your team. **Assign some number of points, possibly 0, to each member of your team based on what you believe was that student's relative participation in the team project, and *explain your scores*.** For allocating points, consider level of effort and contributions for anything relevant to the team project. Note equal participation is not the same thing as equal lines of code produced. It is very important that you explain the scores you give to each student, including yourself: *If you do not provide meaningful explanations, then your response will be considered incomplete and you will receive 0 overall on the assessment.*

For example, in a team where all team members participated more or less equally, the scores would be: Sheldon 25, Leonard 25, Howard 25, Raj 25, and your explanation would describe who did which part(s) of the team project effort.

In a team with a disparity in contributions, however, the scores might be: Homer 40, Marge 25, Bart 20, Lisa 15, and the explanation might describe Homer as doing most of the work followed by a discussion

of a little bit of what the others contributed. Scores like these would most typically be provided by Homer. Marge might instead give these scores: Homer 10, Marge 50, Bart 20, Lisa 20. This is why we need every team members' explanations of who did what, and who did not do what they were expected to do, to fairly credit differential grading.

part 0B (optional): Is there anything that you think should have been covered but wasn't? Is there anything that was covered that shouldn't have been (e.g., everyone already knows from their introductory CS courses)? Is there anything that should have been discussed more, discussed less, or presented in a different way? Is there anything else you would change about the course, including contents, teaching methods, delivery, assignments, anything else you think relevant? Please explain. Suggestions for next year's offering of the course will be greatly appreciated. In addition to submitting comments and suggestions here as part of your second assessment, you can also post anonymously on Edstem. (And please also submit the SEAS evaluation.)

Graded questions: There are three graded questions, parts 1, 2 and 3.

Part 1 (30 points) Reverse-engineer your team's service (not the client app) to produce requirements documentation in the form of *use cases*. Keep in mind that use case actors can be other code or environment resources, and need not be human. You do not need to write or refactor any code. Do not change any code in your team's shared repository(ies). For this question, consider only your service, not your client app. There are two subparts, A and B.

1A (20 points) Write a set of detailed use cases covering the entire functionality of your service that was actually implemented (in your team's main branch) by the end of the second iteration. You should write a *set* of use cases, not a single huge use case. Every use case should have a title, actor(s), description, basic flow, alternative flow(s), trigger(s), precondition(s) and postcondition(s). If for some reason any of these parts are not applicable to one of your use cases, instead give an explanation as to why it's not applicable. Make sure to label the various elements as well as every step in each flow, including the branch points for alternative flows. You will need to refer to the titles and labels in part **1B**.

1B (10 points) Write a table in which you associate each of the major methods in this implementation of your service with the particular labeled parts of the use case(s) they implement. You can omit constructors, getters, setters, utility functions, etc. *unless* that is the only part of your codebase that implements a given part of a use case. That is, every part of each use case from 1A needs to be associated with some code that implements it – otherwise it was not implemented and thus should not be included in your answer to 1A.

One way to make sure you have covered everything is to walk through every method in your codebase and label it with respect to which part of your use cases it implements or decide it is not applicable (e.g., testing code or code that is part of your client app but not part of your service). Keep in mind that some individual method might implement multiple parts of a use case and some parts of a use case might be split among multiple methods. Some methods might be reused in multiple use cases.

Continue to part 2 on next page

Part 2 (35 points): Identify a class or a set of classes in the main branch of your team's codebase (for either your service and/or your client app) that violates at least one of the SOLIDD principles (considering the DRY principle as well as the original five SOLID principles). If your programming language does not have "classes", apply this question instead to the programming language's modular units or files. You do not need to write or refactor any code. Do not change any code in your team's shared repository(ies). For this question, you can consider either your service, or your client app, or both. There are two subparts, A and B.

Part 2A (25 points): State which specific principle is violated (or multiple principles, if applicable) and explain in detail in what way your chosen class (or set of classes) violates that principle. Then explain in detail how you could change that part of your codebase to no longer violate that principle. Your approach should not introduce any new violations of the SOLIDD principles, e.g., do not "fix" a problem in one class by moving the problem to another class!

Part 2B (10 points): Explain in detail how you would check that your hypothetical changes from **2A** would not accidentally change the semantics of your system.

Alternative for part 2A (alternative 25 points): If none of your team's code violates any of the SOLIDD principles, then for each principle pick one or more classes and explain in detail how it aligns with the principle. If a specific principle is simply irrelevant to your codebase, explain in detail why that is the case. For **2B (alternative 10 points)**, just state that it's not applicable since you do not suggest changing anything.

Continue to part 3 on next page

Part 3 (35 points): Did your team use any design patterns in writing the code for your system (in the main branch for either your service and/or your client app)? This might include design patterns listed in any mainstream design patterns catalog whether or not they were discussed in class. You do not need to write or refactor any code. Do not change any code in your team's shared repository(ies). There are two subparts, A and B. For part A of this question, you can consider either your service, or your client app, or both; for part B, please consider only your service.

3A (25 points):

If your team used at least two design patterns (in the main branch for either your service and/or your client app), pick any two distinct design patterns, point to the specific code that reflects each of them, and describe in detail for each case how it implements that design pattern.

If you used only one design pattern, point to the specific code that reflects it and describe in detail how it implements that design pattern. In addition, pick a second design pattern and describe in detail how your team could have used it to improve the quality of your code.

If your team did not use any design patterns, pick two distinct design patterns and describe in detail how your team could have used each of them to improve the quality of your code.

Do not consider MVC as a design pattern, you will receive 0 for this question if you do,

3B (10 points):

Knowing what you know now about software design, including but not limited to software architecture, design patterns, and code structuring, what would you have done differently when you implemented your service (specifically your service, not your client app)?

Your response should include a justification for each of the hypothetical changes you would make to your service (do not actually change any code in your team's shared repository!). Please **avoid** superficial modifications like changing variable or method naming conventions, you will get 0 credit for that kind of change. Instead focus on improvements that could potentially enhance your service's security, performance, accessibility, usability, scalability, availability, extensibility, maintainability, portability, etc.

This is the end of the assessment.

Points 100

Submitting a file upload

File Types pdf, doc, docx, and txt

Due	For	Available from	Until
Dec 11, 2023	Everyone	Dec 8, 2023 at 12:01am	Dec 24, 2023 at 11:59pm

Second Assessment Rubric

You've already rated students with this rubric. Any major changes could affect their assessment results.

Criteria	Ratings	Pts
1A (20 Points) Consider only the 'best' four use cases, with max 5 points each. For each of those use cases, for any part missing without justification, deduct from 5 as below (but do not go below 0): - Title: 0.25 point - Actor: 0.5 point - Clear description: 0.25 point - Precondition: 0.5 point - Postcondition: 0.5 point - Trigger: 0.5 point - Basic flow: 1.5 points - Alternative flow: 1.5 point Alternative flow could be waived if the use case is simple, with no alternatives, error handling, etc.	This area will be used by the assessor to leave comments related to this criterion.	20 pts
1B (10 Points) - Deduct 10/(total number of use cases in 1A) marks for each missing table. - Deduct 2 marks for each table if the corresponding use case is non-trivial (critical to the service) but the table only has trivial functions (constructors, getters, setters, and other utility functions)	This area will be used by the assessor to leave comments related to this criterion.	10 pts
2A (25 Points) grade only one of regular and alternate 2A regular 2A	This area will be used by the assessor to leave comments related to this criterion.	25 pts

Criteria	Ratings	Pts
For each chosen class(es) in the answer: - Deduct 10/(total number of chosen classes) points if the student's explanation is not aligned with the principle For each proposal to change: - Deduct 10/(total number of chosen classes) points if the student's explanation is not aligned with the principle or the proposal is missing alternate 2A Since there are six SOLID principles, counting DRY, consider only 'best' five, with max 5 points each. It's ok if the student omits DRY, then there's only five anyway. - Deduct 5 points if the explanation for each principle is not correct (whether the principle is applied or not applied to student's codebase)		
2B (10 points) - Deduct 4 points if unit tests are not mentioned, give partial marks (up to max 4) for quality of discussion of unit tests - Deduct 3 points if integration tests are not mentioned, give partial marks (up to max 3) for quality of discussion of integration tests - Deduct 3 points if system tests are not mentioned,, give partial marks (up to max 3) for quality of discussion of system tests Give full 10 points if student choose 2A-alternative	This area will be used by the assessor to leave comments related to this criterion.	10 pts
3A (25 Points) Mention MVC as a design pattern: 0 points for 3A. -----	This area will be used by the assessor to leave comments related to this criterion.	25 pts

Criteria	Ratings	Pts
Mention Compound Design Pattern that are not individual design patterns (such as Repository pattern, Service Layer Design Pattern) will be given a maximum of 9 points. The student should mention these or points will be deducted as follows: - Description of the design pattern: What does it do?/ Why does it help? [3 points] - Implementation Details: - Describe the methods (functions) this design pattern has [3 points] - Describe input parameters, returns, and exceptions [3 points] ----- For each of the individual design patterns (2 in total): If the student describes a design pattern that has been implemented. The student should mention these or points will be deducted as follows: - Correct Design Pattern Name [1 point] - Mention Code Location [1 point] - Description of the design pattern: What does it do?/ Why does it help? [3 points] - Implementation Details: - Describe the methods (functions) this design pattern has [3 points] - Describe input parameters, returns, and exceptions [3 points] - Describe edge cases (if applicable) [1.5 points] ----- If the team describes a design pattern that could have been used. The student should mention these or points will be deducted as follows:		

Criteria	Ratings	Pts
- Correct Design Pattern Name [1 point] - Motivation: what does it do?/Why does it help? [3 points] - Implementation Details: - How to refactor the code to include this design pattern [1 point] - Describe the methods (functions) this design pattern has [3 points] - Describe input parameters, returns, and exceptions [3 points] - Describe edge cases (if applicable) [1.5 points]		
3B (10 points) The student should describe one software architecture or design change [10 points]: - What is the software architecture/ design change [1 point] - Motivation for this architecture/ design adaptation [3 points] - Describe what should have been done to implement this architecture instead: - Database changes [3 points] - Codebase changes [3 points]	This area will be used by the assessor to leave comments related to this criterion.	10 pts
Total Points: 100		